Universidad Tecnológica Metropolitana
Facultad de Ingeniería
Escuela de Informática y Computación

Arquitectura de Solución de Inteligencia de Negocios:

Vista Única del Cliente en Transportes Rapiditos S.A. (2023-2025)

Informe de Avance 1

Caso de Negocio, Fuentes de Información y Modelo Multidimensional

Asignatura Inteligencia de Negocios (INFB8101), Sección 412

Proyecto Semestral – II Semestre 2026

Profesor	**Integrantes (Grupo 1)**
Luis Cofré B.	Ignacio Ahumada Poque
	Vicente Fuentes Rodríguez
	Gabriel García Vega
	Leonardo Solís Ortiz

Santiago, Chile, noviembre de 2026

Universidad Tecnológica Metropolitana
Facultad de Ingeniería
Escuela de Informática y Computación

Índice

Resumen Ejecutivo **vi**

1. Caso de Negocio **1**
1.1. Contexto . . . 1
1.2. Organización y Líneas de Negocio . . . 2
1.3. Objetivos . . . 3
1.3.1. Objetivo General . . . 3
1.3.2. Objetivos Específicos . . . 3
1.4. Problemáticas Identificadas . . . 4
1.4.1. Información del Cliente Fragmentada entre Líneas de Negocio . . . 4
1.4.2. Ausencia de un Identificador Común del Cliente . . . 4
1.4.3. Datos Heterogéneos entre Líneas . . . 4
1.4.4. Ausencia de una Visión Anual Consolidada . . . 4
1.4.5. Decisiones Comerciales sin Segmentación de Clientes . . . 5
1.5. Preguntas de Negocio que Orientan la Solución . . . 5
1.6. Perfil de Usuario . . . 6
1.7. Alcances y Limitaciones . . . 7
1.7.1. Alcances . . . 7
1.7.2. Limitaciones . . . 8

2. Fuentes de Información **10**
2.1. Enfoque: Transacciones Públicas y Clientes Sintéticos . . . 10
2.2. Fuentes por Línea de Negocio . . . 11
2.3. Uso de cada Fuente en el Modelo . . . 11
2.4. Fuentes de Apoyo . . . 12
2.5. Tratamientos Previos a la Carga . . . 13
2.6. Estado de Verificación de las Fuentes . . . 14

3. Arquitectura Propuesta **15**
3.1. Componentes y Herramientas . . . 15
3.2. Capas de Almacenamiento . . . 17
3.3. Flujo de Integración (ETL) . . . 17
3.4. Entorno de Ejecución . . . 18

4. Modelamiento Dimensional **19**
4.1. Proceso de Negocio y Grano . . . 19
4.2. Tabla de Hechos . . . 21
4.3. Dimensiones . . . 22
4.3.1. Dimensión Tiempo (`dim_tiempo`) . . . 22
4.3.2. Dimensión Cliente (`dim_cliente`) . . . 22
4.3.3. Dimensión Servicio (`dim_servicio`) . . . 22
4.3.4. Dimensión Zona (`dim_zona`) . . . 22
4.3.5. Dimensión Canal de Venta (`dim_canal_venta`) . . . 22
4.3.6. Dimensión Medio de Pago (`dim_medio_pago`) . . . 23
4.4. Jerarquías . . . 23
4.5. Cálculos y KPI Derivados . . . 23
4.6. Norma de Nomenclatura . . . 24

5. Scripts del Datawarehouse **26**
5.1. Scripts Entregados . . . 26
5.2. Script de Creación del Modelo . . . 26
5.3. Reglas de Integridad . . . 30
5.4. Entorno de Ejecución . . . 30
5.5. Verificación del Datawarehouse . . . 31

6. Bibliografía **36**

7. Anexos **39**
7.1. Anexo A: Diccionario de Datos . . . 39
7.1.1. Tabla `fact_venta` . . . 39
7.1.2. Tabla `dim_tiempo` . . . 40
7.1.3. Tabla `dim_cliente` . . . 41
7.1.4. Tabla `dim_servicio` . . . 42
7.1.5. Tabla `dim_zona` . . . 43
7.1.6. Tabla `dim_canal_venta` . . . 43
7.1.7. Tabla `dim_medio_pago` . . . 43
7.2. Anexo B: Reproducción del Entorno . . . 44
7.2.1. Ejecución de los Scripts en Cualquier Equipo . . . 44
7.2.2. Entorno de Desarrollo del Grupo . . . 44

7.3. Anexo C: Repositorio del Proyecto 45

Índice de Tablas

1.1. Líneas de negocio de Transportes Rapiditos S.A. 2
1.2. Problemáticas identificadas y respuesta de la solución BI 5
1.3. Preguntas de negocio y elementos del modelo que las responden 6
1.4. Perfil de usuario de la reportería 7
1.5. Planificación de entregas del proyecto 8
2.1. Fuente pública de cada línea de negocio 11
2.2. Datos de cada fuente y su destino en el modelo 12
2.3. Fuentes de apoyo y su uso en el proyecto 13
3.1. Herramientas de la arquitectura por capa 16
3.2. Etapas del flujo de integración 18
4.1. Alternativas de modelo evaluadas 19
4.2. Medidas de `fact_venta` 21
4.3. Jerarquías de las dimensiones 23
4.4. Cálculos y KPI derivados de las medidas 24
4.5. Norma de nomenclatura del modelo 25
5.1. Scripts del Datawarehouse y de apoyo 26
5.2. Elementos del esquema `dwh` verificados en el catálogo 32
7.1. Diccionario de datos de `fact_venta` 40
7.2. Diccionario de datos de `dim_tiempo` 41
7.3. Diccionario de datos de `dim_cliente` 42
7.4. Diccionario de datos de `dim_servicio` 42
7.5. Diccionario de datos de `dim_zona` 43
7.6. Diccionario de datos de `dim_canal_venta` 43
7.7. Diccionario de datos de `dim_medio_pago` 44
7.8. Contenido del repositorio del proyecto 46

Índice de Figuras

1.1. Estructura organizacional supuesta de Transportes Rapiditos S.A. 2
3.1. Arquitectura de la solución BI por capas 15
4.1. Modelo multidimensional estrella del esquema `dwh` 20
5.1. Fragmento de `sql/01_ddl.sql`: creación de la tabla de hechos `fact_venta` 28
5.2. Ejecución de `sql/01_ddl.sql` en el Query Tool de pgAdmin 29
5.3. Estructura de la tabla de hechos `fact_venta` en pgAdmin 29
5.4. Salida de `docker compose ps`: PostgreSQL 17 y pgAdmin activos 30
5.5. Esquema estrella en el ERD Tool de pgAdmin 31
5.6. `sql/99_validacion.sql`: tablas del esquema `dwh` 33
5.7. `sql/99_validacion.sql`: llaves foráneas de la tabla de hechos 33
5.8. `sql/99_validacion.sql`: resumen de columnas y llaves por tabla 34
5.9. `sql/99_validacion.sql`: restricciones `CHECK` 34
5.10. `sql/99_validacion.sql`: control de nomenclatura (0 filas fuera de norma) 35

Resumen Ejecutivo

Transportes Rapiditos S.A. es una compañía internacional de transportes con seis líneas de negocio: buses interurbanos, trenes, radio taxis, logística y distribución, aerolíneas y delivery. Cada línea registra sus ventas por separado, por lo que la compañía no puede saber quiénes son sus clientes en conjunto ni cómo se comportan cuando compran en más de una línea. El caso plantea fusionar la información de las líneas para obtener una vista única del cliente y, con ella, determinar qué tipos de clientes tiene la compañía y su comportamiento anual en los últimos tres años.

Para resolverlo, el Grupo 1 propone una Arquitectura de Solución de Inteligencia de Negocios compuesta por una capa de extracción en Python, un proceso ETL en Apache Hop, un Datawarehouse en PostgreSQL 17, un cubo semántico en Cube y un dashboard en Streamlit. La reportería se orienta a un perfil táctico: la gerencia comercial y de marketing.

El Datawarehouse sigue la metodología de Kimball y un modelo estrella. El proceso de negocio son las compras de servicios, y el grano es una fila por cada transacción de compra de un cliente. La tabla de hechos `fact_venta` reúne ocho medidas (cantidad; montos bruto, de descuento y neto; distancia; duración; peso, y edad del cliente) y se relaciona con seis dimensiones: tiempo, cliente, servicio, zona (de origen y de destino), canal de venta y medio de pago. La dimensión cliente es la que unifica al cliente entre las seis líneas.

Como ningún dataset público identifica a un mismo cliente en varias líneas de transporte, el proyecto combina transacciones públicas reales de seis fuentes (una por línea) con una capa de clientes sintéticos residentes en Chile, calibrada con el Censo de Población y Vivienda 2024 del INE. Los datos sintéticos se identifican como tales en el código, en la base de datos y en este informe.

Este Informe de Avance 1 presenta el caso de negocio, las fuentes de información, la arquitectura propuesta, el modelo multidimensional y el script SQL que lo crea, sin datos. El script se ejecutó en PostgreSQL 17.11 y su resultado se verificó con consultas al catálogo y con 29 pruebas automáticas: el esquema `dwh` contiene 7 tablas, 58 columnas, 7 llaves foráneas y 20 reglas de validación, y todas las restricciones e índices cumplen la norma de nomenclatura. El poblamiento mediante ETL y las estadísticas de procesamiento corresponden al Avance N°2; el cubo, los KPI, el dashboard y las recomendaciones, al Avance N°3.

Universidad Tecnológica Metropolitana
Facultad de Ingeniería
Escuela de Informática y Computación

1. Caso de Negocio

1.1. Contexto

El proyecto semestral de la asignatura plantea construir una Arquitectura de Solución de Inteligencia de Negocios (BI) para un caso de negocio. El caso asignado es el siguiente:

> *«Una compañía internacional de transportes decide fusionar sus líneas de negocio en términos de información, para tener una vista única de sus clientes».*

La compañía es **Transportes Rapiditos S.A.**, que transporta pasajeros y carga por tierra y aire mediante seis líneas de negocio. Cada línea vende un servicio distinto: pasajes de bus, tren y avión, viajes de taxi, envíos de logística y pedidos de delivery. Como cada línea registra sus ventas en su propio sistema, la información de un mismo cliente queda repartida entre ellas, y la compañía no puede responder preguntas tan básicas como cuántos clientes tiene en total o cuántos de ellos usan más de un servicio.

Este problema es habitual en las organizaciones con varias unidades de negocio. Kimball y Ross [1] proponen resolverlo con un Datawarehouse cuyas dimensiones sean *conformadas*, es decir, que signifiquen lo mismo para todas las áreas que las usan. En este caso, la dimensión conformada central es el cliente: un mismo cliente debe identificarse igual cuando compra un pasaje de bus, pide un taxi o envía una carga.

El contexto chileno muestra la misma fragmentación. Las estadísticas públicas del sector se publican por modo de transporte y en forma agregada: el Instituto Nacional de Estadísticas (INE) informa el transporte interurbano de pasajeros por carretera [2], la Empresa de los Ferrocarriles del Estado (EFE) informa los pasajeros de sus servicios en su memoria anual [3] y la Junta de Aeronáutica Civil (JAC) publica el tráfico aéreo por ruta y operador [4]. Ninguna de ellas describe al cliente, porque esa información solo existe dentro de cada operador. Por eso la vista única del cliente requiere que la compañía integre sus propios datos.

Los clientes del proyecto residen en Chile. El Censo de Población y Vivienda 2024 del INE censó a 18.480.432 personas en el país [5], y sus distribuciones por comuna, sexo y edad son la base con que se generan los clientes del proyecto [6], como se explica en la Sección 2.

1.2. Organización y Líneas de Negocio

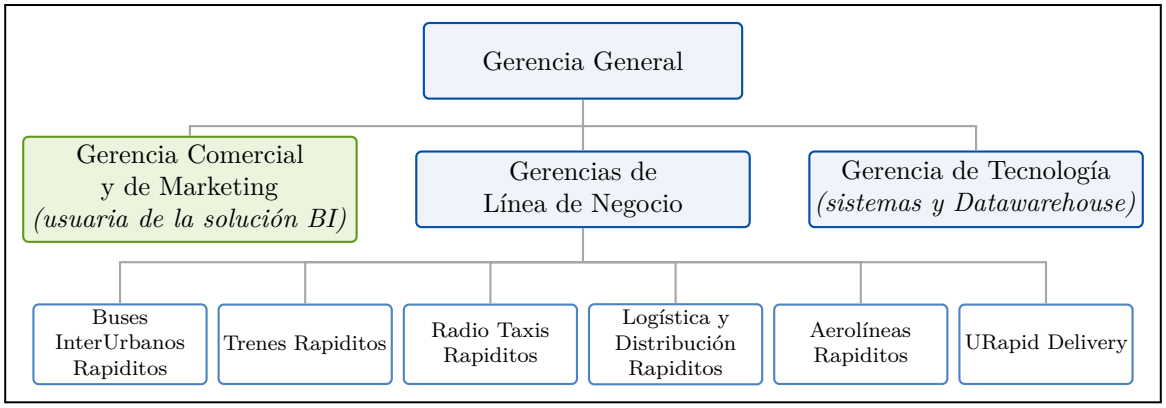
El enunciado define las seis líneas de negocio de la Tabla 1.1. La tabla indica además el medio de transporte, el tipo de servicio y la unidad de venta con que cada línea se representa en el modelo dimensional (dimensión `dim_servicio`).

Tabla 1.1: Líneas de negocio de Transportes Rapiditos S.A.

Línea de negocio	**Medio de transporte**	**Tipo de servicio**	**Unidad de venta**
Buses InterUrbanos Rapiditos	Bus	Pasajeros	Pasaje
Trenes Rapiditos	Tren	Pasajeros	Pasaje
Radio Taxis Rapiditos	Taxi	Pasajeros	Viaje
Logística y Distribución Rapiditos	Camión	Carga	Envío
Aerolíneas Rapiditos	Avión	Pasajeros	Pasaje
URapid Delivery	Moto/bicicleta	Carga	Pedido

El enunciado no describe la organización interna de la compañía. Para situar la solución, el grupo supone la estructura de la Figura 1.1: una gerencia general de la que dependen las gerencias de cada línea de negocio, una gerencia comercial y de marketing transversal, que es la usuaria de la solución BI (Sección 1.6), y una gerencia de tecnología, responsable de los sistemas de cada línea y del Datawarehouse.

Figura 1.1: Estructura organizacional supuesta de Transportes Rapiditos S.A.



Nota. Elaboración propia. La estructura es un supuesto del grupo, porque el enunciado no describe la organización interna de la compañía.

Se asume además que cada línea de negocio opera con su propio sistema de ventas, con sus propios códigos de cliente, de zona y de servicio. Este supuesto es el origen de las problemáticas de la Sección 1.4.

1.3. Objetivos

1.3.1. Objetivo General

Diseñar e implementar una Arquitectura de Solución de Inteligencia de Negocios que integre la información de las seis líneas de negocio de Transportes Rapiditos S.A. en una vista única del cliente, para determinar qué tipos de clientes tiene la compañía y su comportamiento anual en los últimos tres años (2023–2025), según su compra, los medios de transporte que usa, la zona y la edad, entre otras variables.

1.3.2. Objetivos Específicos

1. Identificar y documentar una fuente de información pública para cada línea de negocio, con su contenido, período, licencia y limitaciones.
2. Diseñar un modelo multidimensional tipo estrella, con el proceso de negocio, el grano, las dimensiones y las medidas necesarias para el objetivo, y una norma de nomenclatura aplicada a todo el modelo.
3. Implementar el Datawarehouse en PostgreSQL mediante un script SQL idempotente que cree el modelo y sus reglas de integridad.
4. Desarrollar los procesos ETL en Apache Hop que extraigan, transformen y carguen los datos de las seis líneas en el Datawarehouse sin errores de consistencia, y registrar sus estadísticas de procesamiento.
5. Construir un cubo con las dimensiones, atributos y métricas necesarios para calcular los indicadores clave (KPI) del objetivo.
6. Desarrollar un dashboard de visualización para el perfil táctico de la gerencia comercial y de marketing.
7. Formular recomendaciones de negocio basadas en los resultados del análisis.

1.4. Problemáticas Identificadas

A partir del enunciado y de los supuestos de la sección anterior, se identifican las problemáticas siguientes. Este avance las describe de forma cualitativa; su magnitud se medirá con los datos cargados en el Avance N°2.

1.4.1. Información del Cliente Fragmentada entre Líneas de Negocio

Cada línea conoce solo las compras que el cliente hace en ella. Un cliente que viaja en bus, toma taxis y pide delivery aparece como tres clientes distintos, y ninguna gerencia ve el total de lo que compra ni la combinación de servicios que usa.

1.4.2. Ausencia de un Identificador Común del Cliente

Los sistemas de las líneas no comparten un código de cliente. Sin una llave común no es posible unir las compras de una misma persona o empresa, ni contar cuántos clientes distintos tiene la compañía.

1.4.3. Datos Heterogéneos entre Líneas

Las líneas registran sus datos con criterios distintos, como ocurre también con las fuentes públicas elegidas para el proyecto (Sección 2):

- **Zonas:** cada línea usa su propia división geográfica (terminales, estaciones, zonas de taxi, aeropuertos o ciudades).
- **Monedas y unidades:** los montos están en distintas monedas, y las distancias y los pesos, en distintas unidades (millas o kilómetros; gramos o kilogramos).
- **Codificaciones:** los canales de venta y los medios de pago se nombran de forma diferente en cada sistema.
- **Granularidad temporal:** algunas líneas registran cada transacción con fecha y hora, y otras solo publican totales mensuales o trimestrales.

1.4.4. Ausencia de una Visión Anual Consolidada

Sin datos integrados, la compañía no puede comparar el comportamiento de sus clientes de un año a otro: si crecen o caen sus compras, si cambian de medio de transporte o de canal, o en qué meses se concentra la demanda.

1.4.5. Decisiones Comerciales sin Segmentación de Clientes

Sin conocer los tipos de clientes, las campañas, los descuentos y las ofertas cruzadas entre líneas se diseñan sin saber a quién se dirigen ni qué segmentos generan más ventas.

La Tabla 1.2 resume cada problemática, su consecuencia y el componente de la solución que la aborda.

Tabla 1.2: Problemáticas identificadas y respuesta de la solución BI

Problemática	**Consecuencia**	**Respuesta de la solución**
Información del cliente fragmentada	No se conoce el total de compras de cada cliente	Tabla de hechos única (`fact_venta`) con las ventas de las seis líneas
Sin identificador común	No se pueden contar ni unir los clientes entre líneas	Dimensión `dim_cliente` con una llave natural común (`codigo_cliente`)
Datos heterogéneos	Las cifras de las líneas no son comparables	ETL que normaliza zonas, monedas (CLP), unidades y catálogos
Sin visión anual consolidada	No se observa la evolución de los clientes entre 2023 y 2025	Dimensión `dim_tiempo` con jerarquía hasta el año, y KPI anuales
Decisiones sin segmentación	Campañas y ofertas sin un público objetivo	Cubo y dashboard por tipo de cliente, edad, zona, línea y canal

1.5. Preguntas de Negocio que Orientan la Solución

El objetivo general se descompone en las preguntas de negocio de la Tabla 1.3, agrupadas por las variables que menciona el enunciado. La última columna indica qué elementos del modelo dimensional (Sección 4) permiten responder cada pregunta.

Tabla 1.3: Preguntas de negocio y elementos del modelo que las responden

Tema	**N°**	**Pregunta**	**Elementos del modelo**
Tipos de clientes	P1	¿Qué tipos de clientes tiene la compañía según tipo (persona o empresa), género, rango etario y región de residencia?	`dim_cliente`, `edad_cliente`
	P2	¿Qué proporción de los clientes compra en más de una línea de negocio?	`dim_cliente`, `dim_servicio`
Comportamiento de compra anual	P3	¿Cuánto compra cada tipo de cliente por año y cómo evoluciona entre 2023 y 2025?	`dim_tiempo`, `dim_cliente`, `monto_neto`
	P4	¿Con qué frecuencia compra cada tipo de cliente y cuál es su ticket promedio?	`dim_cliente`, `monto_neto`, conteo de transacciones
	P5	¿En qué meses y trimestres se concentra la demanda de cada línea?	`dim_tiempo`, `dim_servicio`, `cantidad`
Medios de transporte	P6	¿Qué líneas de negocio y medios de transporte prefiere cada segmento de clientes?	`dim_servicio`, `dim_cliente`
Zona	P7	¿Desde qué zonas y hacia qué zonas se mueven los clientes o envían sus cargas?	`dim_zona` (origen y destino)
	P8	¿Cómo varía el gasto según la región de residencia del cliente?	`dim_cliente`, `monto_neto`
Edad	P9	¿Cómo cambia el comportamiento de compra según el rango etario?	`edad_cliente`, `dim_servicio`, `monto_neto`
Canal y pago	P10	¿Por qué canales y con qué medios de pago compra cada tipo de cliente?	`dim_canal_venta`, `dim_medio_pago`
	P11	¿Qué nivel de descuento recibe cada segmento de clientes?	`monto_descuento`, `monto_bruto`

1.6. Perfil de Usuario

El enunciado pide presentar la reportería para un perfil de usuario, operacional/táctico o ejecutivo. El grupo eligió el **perfil táctico**, representado por la **gerencia comercial y de marketing**. La razón es que el objetivo del caso, conocer los tipos de clientes y su

comportamiento anual, sirve sobre todo para decidir a qué segmentos dirigir campañas, descuentos y ofertas entre líneas, que son decisiones de mediano plazo propias de esa gerencia. Un perfil operacional necesitaría el detalle diario de cada línea, y uno ejecutivo, solo unos pocos indicadores globales; ninguno de los dos aprovecha la segmentación de clientes que pide el objetivo.

Tabla 1.4: Perfil de usuario de la reportería

Aspecto	**Descripción**
Usuario	Gerencia comercial y de marketing: gerente, jefaturas de marketing y analistas comerciales
Nivel de decisión	Táctico: planificación de mediano plazo (trimestral y anual)
Decisiones que apoya	Segmentación de clientes, diseño de campañas, política de descuentos, ofertas cruzadas entre líneas y priorización de canales de venta
Frecuencia de consulta	Mensual, con cierres trimestrales y anuales
Nivel de detalle	Agregado por tipo de cliente, rango etario, zona, línea de negocio, canal y período, con desglose hasta mes y región. No requiere el detalle de cada transacción
Indicadores clave	Ventas netas, clientes activos, gasto anual por cliente, frecuencia de compra, ticket promedio, clientes multilínea, participación por línea y tasa de descuento (Sección 4.5)
Herramienta	Dashboard web en Streamlit con filtros por año, línea de negocio, tipo de cliente y zona

1.7. Alcances y Limitaciones

1.7.1. Alcances

El enunciado establece que el modelamiento dimensional debe ser tipo estrella, que la reportería se presenta para un perfil de usuario, que las recomendaciones deben estar asociadas al negocio del caso y que se pueden usar datasets públicos disponibles en la web. Dentro de ese marco, la solución tiene el siguiente alcance:

- **Proceso de negocio:** compras de servicios que realizan los clientes en las seis líneas de negocio.

- **Grano:** una fila por cada transacción de compra de un cliente por un servicio de una línea de negocio, en una fecha: un pasaje de bus, tren o avión, un viaje de taxi, un envío de logística o un pedido de delivery.
- **Período:** los últimos tres años, de 2023 a 2025.
- **Clientes:** personas y empresas residentes en Chile, identificadas con una llave común a todas las líneas.
- **Moneda:** todos los montos se expresan en pesos chilenos (CLP).
- **Usuario:** perfil táctico de la gerencia comercial y de marketing.
- **Tecnología:** Datawarehouse en PostgreSQL 17, ETL en Apache Hop, cubo en Cube y dashboard en Streamlit (Sección 3).

El proyecto se entrega en tres avances, según la Tabla 1.5. Este Informe de Avance 1 incluye el caso de negocio, las fuentes de información, la arquitectura, el modelo multidimensional y el script SQL que crea su estructura, sin datos.

Tabla 1.5: Planificación de entregas del proyecto

Entrega	Fecha	Contenido
Avance N°1	05-11-2026	Caso de negocio, fuentes de información, arquitectura propuesta, modelo multidimensional estrella y script SQL de creación (sin datos).
Avance N°2	19-11-2026	ETL, poblamiento del Datawarehouse y estadísticas de procesamiento.
Avance N°3	10-12-2026	Cubo, KPI, dashboard de visualización, recomendaciones y presentación.

1.7.2. Limitaciones

- **Clientes sintéticos:** ningún dataset público identifica a un mismo cliente en varias líneas de transporte, por lo que los clientes y su comportamiento entre líneas se generan de forma sintética. Los resultados sobre tipos de clientes demuestran el funcionamiento de la solución, pero no describen a clientes reales.
- **Fuentes de otros países:** las transacciones provienen de Brasil, el Reino Unido y Estados Unidos, mientras que los clientes residen en Chile. Esto es coherente con una compañía internacional, pero las zonas de origen y destino no son chilenas.

- **Datos agregados:** ANTT publica totales mensuales de pasajes y DB1B, una muestra trimestral; el detalle por día y por transacción se asigna en el ETL de forma sintética, reproduciendo esos totales.
- **Períodos distintos:** las fuentes que no cubren 2023–2025 se trasladan a ese período, conservando su estacionalidad.
- **Datos ficticios y licencias:** el dataset de trenes es ficticio según su proveedor, y el de Delivery Center no publica su licencia. Si esa licencia no permite su uso, la línea URapid Delivery se generará de forma sintética, sin cambios en el modelo.
- **Alcance de este avance:** el Datawarehouse se entrega sin datos, por lo que las problemáticas no se cuantifican todavía.

2. Fuentes de Información

El enunciado permite usar datasets públicos disponibles en la web. Para cada línea de negocio se eligió una fuente pública con tres criterios: datos lo más cercanos posible a una transacción individual, el período más reciente disponible y una licencia que permita su uso académico.

2.1. Enfoque: Transacciones Públicas y Clientes Sintéticos

Los datos públicos de transporte se publican anonimizados, por lo que ningún dataset identifica a un mismo cliente en distintas líneas de negocio. Como el caso exige una vista única del cliente, el proyecto combina dos tipos de datos:

- **Transacciones públicas:** los viajes, pasajes, envíos y pedidos provienen de las fuentes de la Tabla 2.1.
- **Clientes sintéticos:** las identidades de los clientes (RUT ficticio, género, fecha de nacimiento, tipo de cliente y comuna de residencia) se generan de forma sintética. Los clientes residen en Chile, y su distribución por comuna, sexo y edad se calibra con el Censo de Población y Vivienda 2024 del INE [6]. Cada transacción pública se asigna a uno de estos clientes, de modo que un mismo cliente puede comprar en varias líneas.

Todo dato sintético se identifica como tal en el código del ETL, en la base de datos y en este informe.

2.2. Fuentes por Línea de Negocio

Tabla 2.1: Fuente pública de cada línea de negocio

Línea	Fuente (publicador, país)	Detalle	Período	Licencia
Buses InterUrbanos	Monitriip «Bilhetes de Passagem», Agência Nacional de Transportes Terrestres (ANTT), Brasil [7]	Pasajes de buses interestatales por mes, origen, destino y tipo de servicio	2019–2025, mensual	Creative Commons Atribución
Trenes	«UK Train Rides», Maven Analytics, Reino Unido [8]	Un registro por pasaje; datos ficticios según el proveedor	Ene.–abr. 2024	Dominio público
Radio Taxis	TLC Trip Record Data, NYC Taxi & Limousine Commission, EE.UU. [9]	Un registro por viaje	2009–2026, mensual	Términos de uso de NYC.gov
Logística y Distribución	«Brazilian E-Commerce Public Dataset», Olist, Brasil [10]	Un registro por pedido e ítem	2016–2018	CC BY-NC-SA 4.0
Aerolíneas	Airline Origin and Destination Survey (DB1B), Bureau of Transportation Statistics, EE.UU. [11]	Muestra de pasajes aéreos	Trimestral, incluye 2023–2025	Datos públicos del gobierno de EE.UU.
URapid Delivery	«Delivery Center: Food & Goods orders», Delivery Center, Brasil [12, 13]	Un registro por pedido	Ene.–abr. 2021	No publicada

Nota. Elaboración propia a partir de las páginas de cada fuente, consultadas en noviembre de 2026. Para logística se mantiene como alternativa el dataset DataCo Smart Supply Chain, con licencia CC BY 4.0 [14].

La compañía del caso es internacional. Por eso, que las fuentes provengan de distintos países es coherente con la dimensión `dim_zona`, que registra país y continente.

2.3. Uso de cada Fuente en el Modelo

La Tabla 2.2 indica qué datos aporta cada fuente y a qué parte del modelo estrella llegan.

Tabla 2.2: Datos de cada fuente y su destino en el modelo

Línea	**Datos que aporta**	**Destino en el modelo**
Buses InterUrbanos	Origen y destino, tipo de servicio, cantidad de pasajes, valor medio, gratuidades	`dim_zona` (origen y destino), `dim_servicio`, `cantidad`, montos
Trenes	Estaciones, fecha y hora, precio, tipo de compra (en línea o en estación), medio de pago, clase	`dim_zona`, `dim_tiempo`, montos, `dim_canal_venta`, `dim_medio_pago`, `dim_servicio`
Radio Taxis	Zonas de origen y destino, horas de inicio y fin, distancia, tarifas, tipo de pago	`dim_zona`, `dim_tiempo`, `duracion_minutos`, `distancia_km`, montos, `dim_medio_pago`
Logística y Distribución	Ciudad del vendedor y del cliente, precio, costo de envío, peso del producto, tipo de pago	`dim_zona`, montos, `peso_kg`, `dim_medio_pago`
Aerolíneas	Aeropuertos de origen y destino, tarifa, pasajeros, trimestre	`dim_zona`, montos, `cantidad`, `dim_tiempo`
URapid Delivery	Canal, medio de pago, monto del pedido, tarifa de envío	`dim_canal_venta`, `dim_medio_pago`, montos

2.4. Fuentes de Apoyo

Además de las fuentes de transacciones, el proyecto usa las fuentes de apoyo de la Tabla 2.3. La dimensión `dim_tiempo` no requiere fuente externa: se genera con un script SQL.

Tabla 2.3: Fuentes de apoyo y su uso en el proyecto

Fuente	**Uso**	**Se carga al DWH**
Censo de Población y Vivienda 2024, INE [6]	Calibrar los clientes sintéticos y poblar sus comunas y regiones de residencia	Sí, en `dim_cliente`
Tablas de zonas de cada fuente	Zonas de taxi de Nueva York, puntos de origen y destino de ANTT, estaciones del Reino Unido y aeropuertos de EE.UU.	Sí, en `dim_zona`
Tipos de cambio del Banco Central de Chile [15]	Expresar en pesos chilenos (CLP) los montos en dólares, libras y reales	Sí, en los montos
Encuesta de transporte interurbano de pasajeros, INE [2]; memoria anual de EFE [3]; estadísticas de la JAC [4]	Contexto chileno para el análisis y las recomendaciones	No

2.5. Tratamientos Previos a la Carga

Antes de cargar los datos en el Datawarehouse, el ETL aplica los tratamientos siguientes. Las limitaciones que estos tratamientos implican se detallan en la Sección 1.7.2.

1. **Clientes sintéticos:** generación de los clientes residentes en Chile y asignación de cada transacción a un cliente.
2. **Desagregación:** las fuentes que publican totales (ANTT por mes y ruta, DB1B por trimestre) se convierten en transacciones individuales sintéticas que reproducen esos totales.
3. **Período de análisis:** el objetivo pide los últimos tres años, por lo que se trabaja con 2023 a 2025. Las fuentes de otros años (Olist, Delivery Center) o que cubren solo una parte del período (UK Train Rides) trasladan sus fechas a ese rango, conservando su estacionalidad.
4. **Unidades y moneda:** millas a kilómetros, gramos a kilogramos y montos a CLP.
5. **Muestreo:** de las fuentes con millones de registros, como los viajes de taxi de Nueva York, se carga una muestra.

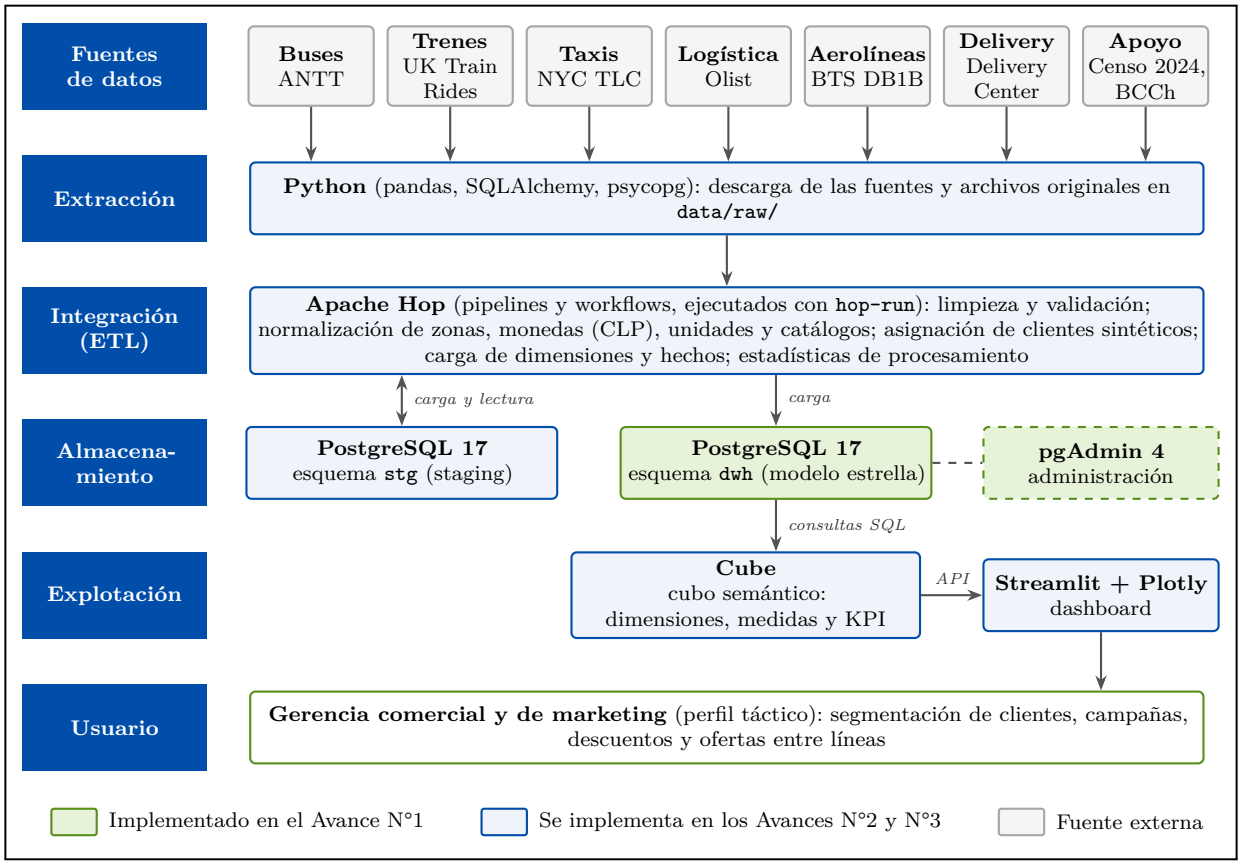
2.6. Estado de Verificación de las Fuentes

Las fuentes se revisaron en noviembre de 2026. Los datos de UK Train Rides, NYC TLC y DataCo se verificaron en sus páginas oficiales. De ANTT, Olist y BTS solo se vieron extractos en resultados de búsqueda (el portal de ANTT bloquea el acceso automatizado y la página de BTS respondió con error), y la página original de Delivery Center no se encontró: se revisó una copia publicada en GitHub. Antes de la carga del Avance N°2 se confirmarán en el navegador las columnas, la granularidad y la licencia de estas fuentes, además de la serie de tipos de cambio que se usará.

3. Arquitectura Propuesta

La solución sigue una arquitectura por capas, desde las fuentes de datos hasta el usuario final (Figura 3.1). Los datos se extraen de las fuentes públicas con Python, se integran con procesos ETL en Apache Hop y se almacenan en un Datawarehouse en PostgreSQL. Desde ahí, un cubo semántico en Cube define las dimensiones y métricas que consume el dashboard en Streamlit, dirigido a la gerencia comercial y de marketing. Todos los servicios corren en contenedores Docker, por lo que el entorno se reproduce igual en Linux y en Windows.

Figura 3.1: Arquitectura de la solución BI por capas



Nota. Elaboración propia. BCCh: Banco Central de Chile (tipos de cambio). Todos los servicios se ejecutan en contenedores Docker Compose.

3.1. Componentes y Herramientas

La Tabla 3.1 detalla las herramientas de cada capa. Las versiones son las instaladas en el equipo de desarrollo y quedan fijadas en `docker-compose.yml` (imágenes de los servicios) y

en `requirements.txt` (paquetes de Python).

Tabla 3.1: Herramientas de la arquitectura por capa

Capa	**Herramienta**	**Versión**	**Función**
Extracción	Python	3.13.5	Lenguaje de los scripts de extracción y de apoyo
	pandas	3.1.0	Lectura y preparación de archivos CSV, Excel y Parquet
	SQLAlchemy y psycopg	2.1.6 y 3.3.8	Conexión de Python con PostgreSQL
Integración	Apache Hop [16]	2.20.0	Pipelines y workflows ETL, ejecutados por línea de comandos (`hop-run`); requiere Java 17 o superior
Almacenamiento	PostgreSQL [17]	17.11	Motor del Datawarehouse (esquemas `stg` y `dwh`)
	pgAdmin 4 [18]	9.18.0	Administración web de la base, Query Tool y ERD Tool
	Docker y Docker Compose [19]	26.1.5 y 2.26.1	Contenedores de los servicios
Explotación	Cube (Cube Core) [20]	1.8.19	Cubo semántico: dimensiones y métricas definidas en YAML
	Streamlit [21] y Plotly	1.66.1 y 7.1.0	Dashboard de visualización web
Calidad y documentación	pytest y ruff	9.2.0 y 0.17.3	Pruebas automáticas y revisión de estilo del código
	Playwright	1.64.1	Capturas automatizadas de pgAdmin para el informe
	Git y GitHub	—	Control de versiones del proyecto
	LaTeX (LuaLaTeX)	TeX Live 2026	Redacción de este informe

Nota. Versiones instaladas y verificadas en el equipo de desarrollo, salvo Apache Hop y Cube, que se instalan en el Avance N°2 (ETL) y el Avance N°3 (cubo). Para ellas se fijaron las últimas versiones estables publicadas al 5 de noviembre de 2026: Apache Hop 2.20.0 (`downloads.apache.org`) y la imagen `cubejs/cube:v1.8.19` (Docker Hub).

3.2. Capas de Almacenamiento

Los datos pasan por cuatro lugares de almacenamiento antes de llegar al usuario:

1. **`data/raw/`:** archivos originales descargados de cada fuente, sin modificar. Permiten repetir el ETL desde cero.
2. **`data/processed/`:** archivos intermedios, como la muestra de viajes de taxi o los clientes sintéticos generados.
3. **Esquema `stg` (staging):** tablas con los datos de cada fuente tal como llegan, donde el ETL los limpia y normaliza. Se agrega en el Avance N°2.
4. **Esquema `dwh`:** el modelo estrella definitivo (Sección 4), que solo recibe datos validados. Está implementado en este avance.

Separar `stg` de `dwh` evita que un error de transformación deje datos inconsistentes en el modelo: si una carga falla, el esquema `dwh` no cambia.

3.3. Flujo de Integración (ETL)

La Tabla 3.2 resume las etapas del proceso de integración que se construirá en el Avance N°2. El número de registros leídos, cargados y rechazados en cada etapa formará la sección Estadísticas de Procesamiento.

Tabla 3.2: Etapas del flujo de integración

N°	Etapa	Herramienta	Descripción
1	Extracción	Python	Descarga de las fuentes a `data/raw/` y muestreo de las fuentes de gran volumen
2	Clientes sintéticos	Python	Generación de los clientes residentes en Chile con las distribuciones del Censo 2024
3	Carga a staging	Apache Hop	Lectura de los archivos y carga en el esquema `stg`
4	Transformación	Apache Hop	Limpieza, conversión de unidades y moneda, traslado de fechas a 2023–2025, normalización de zonas, canales y medios de pago, y asignación de cada transacción a un cliente
5	Carga de dimensiones	Apache Hop	Inserción de los registros nuevos en las seis dimensiones, buscando por su llave natural (`codigo_<entidad>`)
6	Carga de hechos	Apache Hop	Búsqueda de las llaves sustitutas de cada dimensión e inserción en `fact_venta`
7	Validación	Apache Hop y SQL	Conteo de registros por etapa, registros rechazados y comparación de totales con las fuentes

3.4. Entorno de Ejecución

El archivo `docker-compose.yml` define los servicios de la solución con imágenes de versión fija: PostgreSQL (`postgres:17.11`) y pgAdmin (`dpage/pgadmin4:9.18.0`); Cube (`cubejs/cube:v1.8.19`) se agregará en el Avance N°3. Los puertos se publican solo en la interfaz local (`127.0.0.1`), los datos de cada servicio se guardan en volúmenes de Docker, y las credenciales se leen desde un archivo `.env` que no se versiona. Con estas decisiones, cualquier integrante levanta el mismo entorno con un solo comando, en Linux o en Windows (Anexo B, Sección 7.2).

El código del proyecto se versiona con Git en un repositorio público de GitHub (Anexo C, Sección 7.3). Los scripts de Python se revisan con ruff y el Datawarehouse se valida con pruebas automáticas en pytest (Sección 5.5).

4. Modelamiento Dimensional

El modelo se diseñó con la metodología de Kimball y Ross [1], que define cuatro pasos en este orden: elegir el proceso de negocio, declarar el grano, identificar las dimensiones e identificar los hechos (medidas). El resultado es el modelo estrella de la Figura 4.1: la tabla de hechos `fact_venta` al centro y las seis dimensiones alrededor. Ninguna dimensión referencia a otra, como exige el modelo estrella.

4.1. Proceso de Negocio y Grano

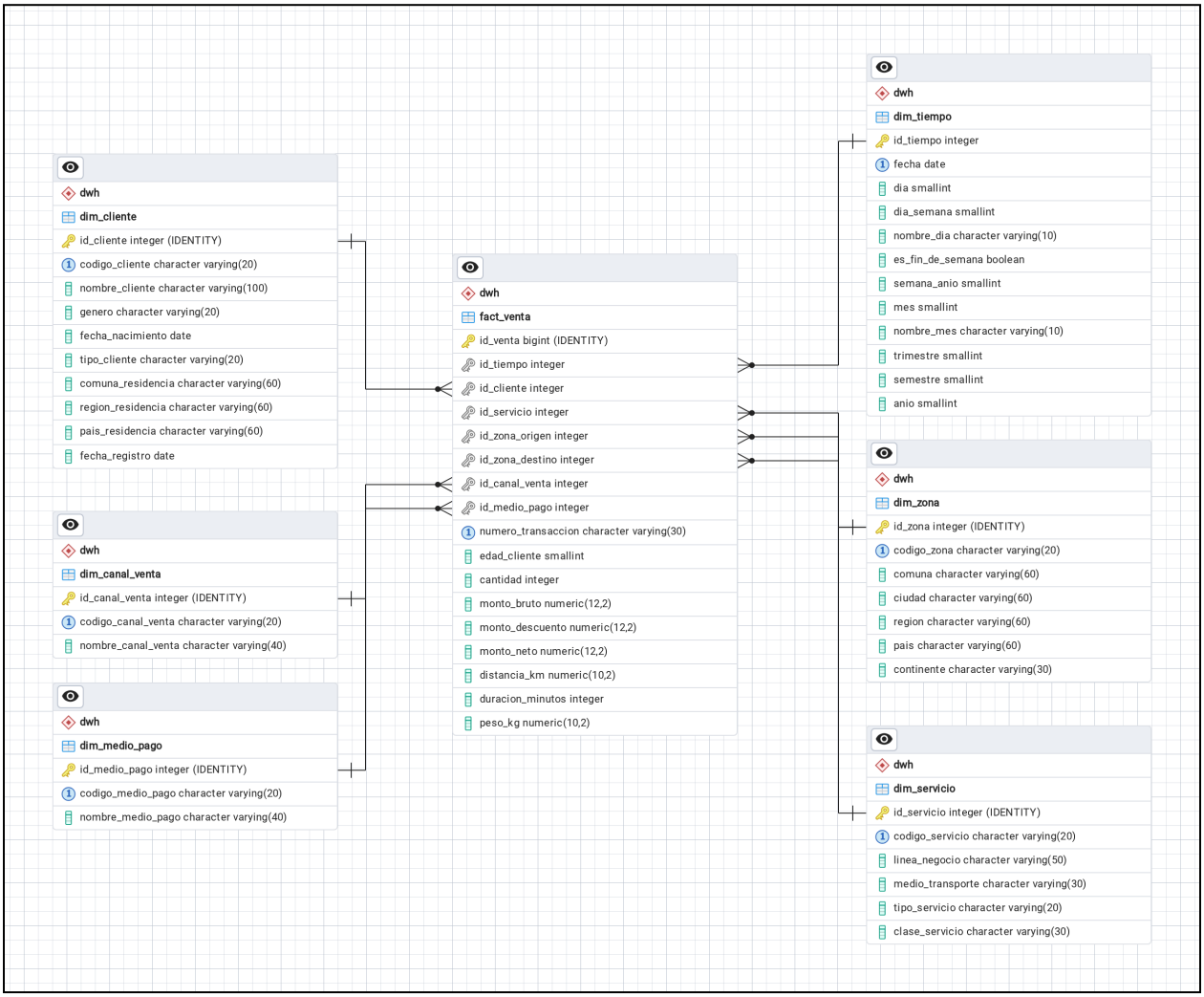
- **Proceso de negocio:** compras de servicios realizadas por los clientes en las seis líneas de negocio.
- **Grano:** una fila por cada transacción de compra de un cliente por un servicio de una línea de negocio, en una fecha.

Se eligió el grano más fino disponible, la transacción, porque permite agregar los datos a cualquier nivel que necesite el análisis (cliente, mes, año, zona o línea) y conserva todas las variables de comportamiento del objetivo. Antes de elegirlo se evaluaron las tres alternativas de la Tabla 4.1.

Tabla 4.1: Alternativas de modelo evaluadas

Criterio	**A. Hechos por transacción (elegida)**	**B. Snapshot mensual cliente-línea**	**C. Constelación (un hecho por línea)**
Ajuste al enunciado (estrella, vista única del cliente)	Alto	Alto	Bajo: deja de ser una estrella única
Análisis por compra, medio de transporte, zona y edad	Completo	Parcial: pierde origen, destino y canal	Completo pero repartido
Simplicidad de ETL y script	Media	Alta	Baja
Riesgo de no cubrir las medidas necesarias	Bajo	Alto	Medio

Figura 4.1: Modelo multidimensional estrella del esquema `dwh`



Nota. Diagrama generado con el ERD Tool de pgAdmin 4 a partir de la base de datos creada por `sql/01_ddl.sql`.

Se eligió la alternativa A porque es una estrella única centrada en el cliente y conserva todas las variables de comportamiento que pide el objetivo.

4.2. Tabla de Hechos

La tabla `fact_venta` registra cada transacción de compra. Contiene:

- **Siete llaves foráneas** hacia las dimensiones: `id_tiempo`, `id_cliente`, `id_servicio`, `id_zona_origen`, `id_zona_destino`, `id_canal_venta` e `id_medio_pago`.
- **Una dimensión degenerada:** `numero_transaccion`, el identificador de la transacción en el sistema de origen, prefijado por línea de negocio para que sea único entre líneas.
- **Ocho medidas**, descritas en la Tabla 4.2.

Tabla 4.2: Medidas de `fact_venta`

Medida	Tipo de dato	Tipo de medida	Descripción
`cantidad`	INTEGER	Aditiva	Pasajes, bultos o unidades
`monto_bruto`	NUMERIC(12,2)	Aditiva	Monto antes de descuento (CLP)
`monto_descuento`	NUMERIC(12,2)	Aditiva	Descuento aplicado (CLP)
`monto_neto`	NUMERIC(12,2)	Aditiva	Monto pagado (CLP): bruto menos descuento
`distancia_km`	NUMERIC(10,2)	Aditiva	Distancia recorrida
`duracion_minutos`	INTEGER	Aditiva	Duración del servicio
`peso_kg`	NUMERIC(10,2)	Aditiva	Peso de la carga; solo logística y delivery
`edad_cliente`	SMALLINT	No aditiva	Edad al momento de la compra; nula para empresas

La edad se guarda en la tabla de hechos, y no solo como fecha de nacimiento en `dim_cliente`, porque el objetivo analiza el comportamiento anual: un mismo cliente cambia de rango etario a lo largo de los tres años, y cada compra debe quedar asociada a la edad que tenía en ese momento. Como no es aditiva, la edad se analiza con promedios o como rango etario, nunca sumada.

4.3. Dimensiones

Todas las dimensiones tienen una llave sustituta generada por la base de datos (`id_<entidad>`, de tipo `IDENTITY`) y una llave natural única (`codigo_<entidad>`), que es la que usa el ETL para no duplicar registros. La excepción es `dim_tiempo`, cuya llave es la fecha en formato AAAAMMDD. El detalle de cada columna está en el Anexo A (Sección 7.1).

4.3.1. Dimensión Tiempo (`dim_tiempo`)

Permite el análisis anual que pide el objetivo y su desglose por semestre, trimestre, mes, semana y día. Tiene granularidad diaria y atributos de fecha, día, día de la semana, nombre del día, indicador de fin de semana, semana del año, mes, nombre del mes, trimestre, semestre y año. Se genera con un script SQL para todo el período 2023–2025.

4.3.2. Dimensión Cliente (`dim_cliente`)

Es la dimensión central del caso, porque unifica al cliente entre las seis líneas: su llave natural (`codigo_cliente`, el RUT o documento) es la misma en todas ellas. Registra nombre, género, fecha de nacimiento, tipo de cliente (persona o empresa), comuna, región y país de residencia, y fecha de registro. En este proyecto sus registros son sintéticos (Sección 2).

4.3.3. Dimensión Servicio (`dim_servicio`)

Describe qué compró el cliente: la línea de negocio, el medio de transporte, el tipo de servicio (pasajeros o carga) y la clase de servicio (por ejemplo, económica, ejecutiva, express o estándar). La línea de negocio y el medio de transporte se guardan en esta misma tabla, y no en dimensiones aparte, para mantener la estrella sin dimensiones que referencien a otras.

4.3.4. Dimensión Zona (`dim_zona`)

Describe el lugar de origen y de destino del servicio, con comuna, ciudad, región, país y continente. Es una dimensión de *rol doble*: la tabla de hechos la referencia dos veces, con `id_zona_origen` e `id_zona_destino`, en lugar de duplicar la tabla.

4.3.5. Dimensión Canal de Venta (`dim_canal_venta`)

Registra el canal por el que se hizo la compra, como aplicación móvil, sitio web, boletería, call center o convenio con empresas.

4.3.6. Dimensión Medio de Pago (`dim_medio_pago`)

Registra el medio con que se pagó la compra, como efectivo, tarjeta de débito, tarjeta de crédito, transferencia o billetera digital.

4.4. Jerarquías

Las dimensiones tienen las jerarquías de la Tabla 4.3, que permiten al usuario agregar los datos o desglosarlos desde el nivel más general al más detallado en el cubo y el dashboard.

Tabla 4.3: Jerarquías de las dimensiones

Dimensión	**Jerarquía (de general a detallado)**	**Uso en el análisis**
`dim_tiempo`	Año → semestre → trimestre → mes → fecha	Comportamiento anual y estacionalidad
`dim_zona`	Continente → país → región → ciudad → comuna	Zonas de origen y destino
`dim_cliente`	País → región → comuna de residencia	Clientes y gasto por lugar de residencia
`dim_servicio`	Tipo de servicio → línea de negocio → clase de servicio	Uso de líneas y medios de transporte

4.5. Cálculos y KPI Derivados

Las medidas de la tabla de hechos permiten calcular los indicadores de la Tabla 4.4, que responden las preguntas de negocio de la Sección 1.5. Se definirán como métricas del cubo en el Avance N°3. El rango etario se calcula a partir de `edad_cliente` en el cubo.

Tabla 4.4: Cálculos y KPI derivados de las medidas

KPI	**Cálculo**	**Preguntas**
Ventas netas	`SUM(monto_neto)`	P3, P8
Ticket promedio	`SUM(monto_neto) / COUNT(id_venta)`	P4
Clientes activos por año	`COUNT(DISTINCT id_cliente)` por año	P1, P3
Frecuencia de compra	`COUNT(id_venta) / COUNT(DISTINCT id_cliente)`	P4
Gasto anual por cliente	`SUM(monto_neto) / COUNT(DISTINCT id_cliente)` por año	P3, P9
Participación por línea	`SUM(monto_neto)` de la línea / `SUM(monto_neto)` total	P5, P6
Clientes multilínea	Clientes con `COUNT(DISTINCT linea_negocio) > 1` por año	P2
Tasa de descuento	`SUM(monto_descuento) / SUM(monto_bruto)`	P11

Nota. Las preguntas P7 y P10 se responden con los mismos indicadores agrupados por zona de origen y destino, canal de venta y medio de pago.

4.6. Norma de Nomenclatura

Todo el modelo sigue la norma de la Tabla 4.5. La consulta de control del script `sql/99_validacion.sql` verifica que ninguna restricción ni índice quede fuera de ella (Sección 5.5).

Tabla 4.5: Norma de nomenclatura del modelo

Elemento	**Regla**	**Ejemplo**
Tablas	Prefijo `fact_` o `dim_`; singular, `snake_case` y minúsculas	`fact_venta`, `dim_cliente`
Columnas	Singular, `snake_case` y minúsculas	`monto_neto`
Llave primaria	`id_<entidad>`	`id_cliente`
Llave natural	`codigo_<entidad>`	`codigo_cliente`
Llave foránea	Mismo nombre que la llave primaria referenciada, con sufijo de rol si corresponde	`id_zona_origen`
Restricción PK	`pk_<tabla>`	`pk_fact_venta`
Restricción FK	`fk_<tabla>_<columna>`	`fk_fact_venta_id_cliente`
Restricción UNIQUE	`uq_<tabla>_<columna>`	`uq_dim_zona_codigo_zona`
Restricción CHECK	`ck_<tabla>_<columna>`, con la columna que valida	`ck_fact_venta_monto_neto`
Índice	`ix_<tabla>_<columna>`	`ix_fact_venta_id_tiempo`

5. Scripts del Datawarehouse

Este capítulo presenta los scripts que crean y verifican el Datawarehouse. En el Avance N°1 el script crea la estructura del modelo multidimensional *sin datos*; los scripts de carga se agregan en el Avance N°2.

5.1. Scripts Entregados

Tabla 5.1: Scripts del Datawarehouse y de apoyo

Archivo	**Propósito**
`sql/01_ddl.sql`	Crea el esquema `dwh` con las seis dimensiones, la tabla de hechos, sus restricciones, índices y comentarios
`sql/99_validacion.sql`	Consulta el catálogo de PostgreSQL para verificar tablas, columnas, llaves, restricciones, índices y nomenclatura
`tests/test_ddl.py`	29 pruebas automáticas (pytest) que crean una base de prueba, ejecutan el script dos veces y comparan el resultado con el diseño
`docker-compose.yml`	Define los servicios PostgreSQL 17 y pgAdmin 4
`herramientas/capturar_pgadmin.py`	Ejecuta el script y la validación en pgAdmin y toma las capturas del informe (Playwright)
`herramientas/generar_diccionario.py`	Genera el diccionario de datos del Anexo A desde el catálogo de la base

5.2. Script de Creación del Modelo

El script `sql/01_ddl.sql` está escrito para PostgreSQL 17 y se ejecuta de corrido, sin intervención, tanto con `psql` como en el Query Tool de pgAdmin. Su estructura es la siguiente:

1. **Codificación:** `SET client_encoding = 'UTF8'`. El archivo está en UTF-8, y esta línea asegura que las tildes de los comentarios y de las reglas se guarden bien aunque el script se ejecute con `psql` desde una consola de Windows, que usa otra codificación.

2. **Inicio de la transacción:** `BEGIN`, seguido de `DROP SCHEMA IF EXISTS dwh CASCADE` y `CREATE SCHEMA dwh`. El script es *idempotente*: se puede ejecutar las veces que sea necesario y siempre deja el mismo resultado.

3. **Dimensiones:** las seis tablas `dim_`, cada una con su llave primaria, su llave natural única

y sus reglas `CHECK`.

4. **Tabla de hechos:** `fact_venta`, con sus siete llaves foráneas (reglas `ON DELETE RESTRICT` y `ON UPDATE RESTRICT`) y las reglas de sus medidas.
5. **Comentarios:** `COMMENT ON` en el esquema, las 7 tablas y las 58 columnas (66 comentarios), que documentan el modelo dentro de la propia base.
6. **Índices:** uno por cada llave foránea de `fact_venta`, porque PostgreSQL no los crea por su cuenta y aceleran los cruces con las dimensiones.
7. **Fin de la transacción:** `COMMIT`. Si cualquier sentencia falla, la transacción se revierte completa y no queda nada a medias.

La Figura 5.1 muestra la creación de la tabla de hechos. El script completo se entrega en el repositorio del proyecto.

Figura 5.1: Fragmento de `sql/01_ddl.sql`: creación de la tabla de hechos `fact_venta`

```
212 CREATE TABLE dwh.fact_venta (
213     id_venta            BIGINT         GENERATED BY DEFAULT AS IDENTITY,
214     id_tiempo           INTEGER        NOT NULL,
215     id_cliente          INTEGER        NOT NULL,
216     id_servicio         INTEGER        NOT NULL,
217     id_zona_origen      INTEGER        NOT NULL,
218     id_zona_destino     INTEGER        NOT NULL,
219     id_canal_venta      INTEGER        NOT NULL,
220     id_medio_pago       INTEGER        NOT NULL,
221     numero_transaccion  VARCHAR(30)    NOT NULL,
222     edad_cliente        SMALLINT       NULL,
223     cantidad            INTEGER        NOT NULL DEFAULT 1,
224     monto_bruto         NUMERIC(12,2)  NOT NULL,
225     monto_descuento     NUMERIC(12,2)  NOT NULL DEFAULT 0,
226     monto_neto          NUMERIC(12,2)  NOT NULL,
227     distancia_km        NUMERIC(10,2)  NULL,
228     duracion_minutos    INTEGER        NULL,
229     peso_kg             NUMERIC(10,2)  NULL,
230     CONSTRAINT pk_fact_venta PRIMARY KEY (id_venta),
231     CONSTRAINT uq_fact_venta_numero_transaccion UNIQUE (numero_transaccion),
232     CONSTRAINT fk_fact_venta_id_tiempo FOREIGN KEY (id_tiempo)
233         REFERENCES dwh.dim_tiempo (id_tiempo) ON DELETE RESTRICT ON UPDATE RESTRICT,
234     CONSTRAINT fk_fact_venta_id_cliente FOREIGN KEY (id_cliente)
235         REFERENCES dwh.dim_cliente (id_cliente) ON DELETE RESTRICT ON UPDATE RESTRICT,
236     CONSTRAINT fk_fact_venta_id_servicio FOREIGN KEY (id_servicio)
237         REFERENCES dwh.dim_servicio (id_servicio) ON DELETE RESTRICT ON UPDATE RESTRICT,
238     CONSTRAINT fk_fact_venta_id_zona_origen FOREIGN KEY (id_zona_origen)
239         REFERENCES dwh.dim_zona (id_zona) ON DELETE RESTRICT ON UPDATE RESTRICT,
240     CONSTRAINT fk_fact_venta_id_zona_destino FOREIGN KEY (id_zona_destino)
241         REFERENCES dwh.dim_zona (id_zona) ON DELETE RESTRICT ON UPDATE RESTRICT,
242     CONSTRAINT fk_fact_venta_id_canal_venta FOREIGN KEY (id_canal_venta)
243         REFERENCES dwh.dim_canal_venta (id_canal_venta) ON DELETE RESTRICT ON UPDATE RESTRICT,
244     CONSTRAINT fk_fact_venta_id_medio_pago FOREIGN KEY (id_medio_pago)
245         REFERENCES dwh.dim_medio_pago (id_medio_pago) ON DELETE RESTRICT ON UPDATE RESTRICT,
246     CONSTRAINT ck_fact_venta_edad_cliente CHECK (edad_cliente BETWEEN 0 AND 120),
247     CONSTRAINT ck_fact_venta_cantidad CHECK (cantidad >= 1),
248     CONSTRAINT ck_fact_venta_monto_bruto CHECK (monto_bruto >= 0),
249     CONSTRAINT ck_fact_venta_monto_descuento CHECK (
250         monto_descuento >= 0 AND monto_descuento <= monto_bruto),
251     CONSTRAINT ck_fact_venta_monto_neto CHECK (monto_neto = monto_bruto - monto_descuento),
252     CONSTRAINT ck_fact_venta_distancia_km CHECK (distancia_km >= 0),
253     CONSTRAINT ck_fact_venta_duracion_minutos CHECK (duracion_minutos >= 0),
254     CONSTRAINT ck_fact_venta_peso_kg CHECK (peso_kg >= 0)
255 );
```

La Figura 5.2 muestra la ejecución del script en el Query Tool de pgAdmin, y la Figura 5.3 la estructura de la tabla de hechos resultante.

Figura 5.2: Ejecución de `sql/01_ddl.sql` en el Query Tool de pgAdmin

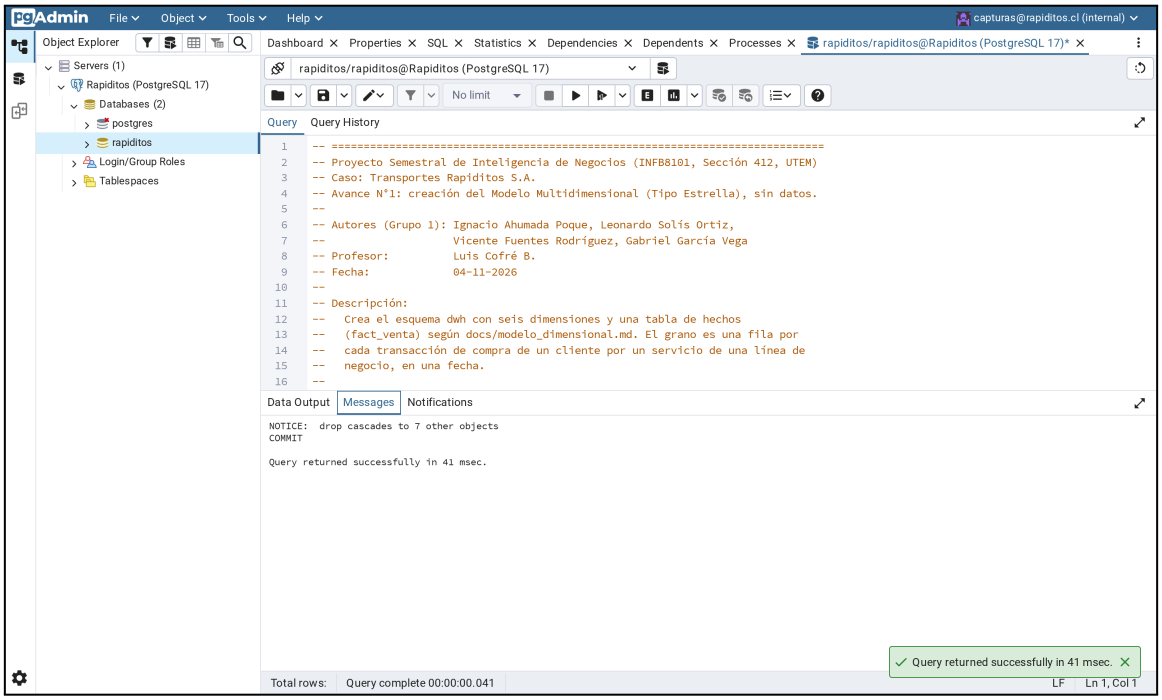
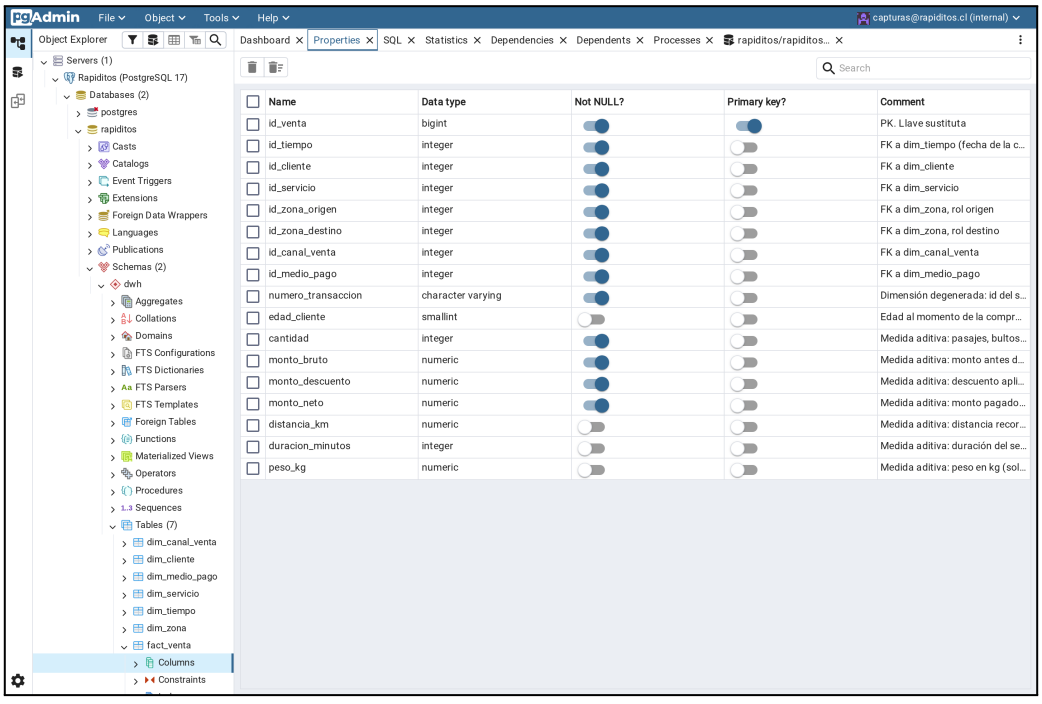


Figura 5.3: Estructura de la tabla de hechos `fact_venta` en pgAdmin



5.3. Reglas de Integridad

Además de las llaves, el script declara 20 restricciones `CHECK` que la base de datos hace cumplir en cada carga. Así, el ETL no puede dejar datos inconsistentes en el Datawarehouse:

- **Montos:** el monto neto es igual al bruto menos el descuento, y el descuento está entre cero y el monto bruto.
- **Medidas no negativas:** la cantidad es de al menos una unidad; distancia, duración y peso no pueden ser negativos, y la edad del cliente está entre 0 y 120 años.
- **Dimensión tiempo:** la llave AAAAMMDD y cada atributo derivado (día, día de la semana, fin de semana, semana del año, mes, trimestre, semestre y año) deben coincidir con la fecha.
- **Dominios cerrados:** el tipo de cliente (persona o empresa), el tipo de servicio (pasajeros o carga) y la línea de negocio, que solo acepta los seis nombres del enunciado.

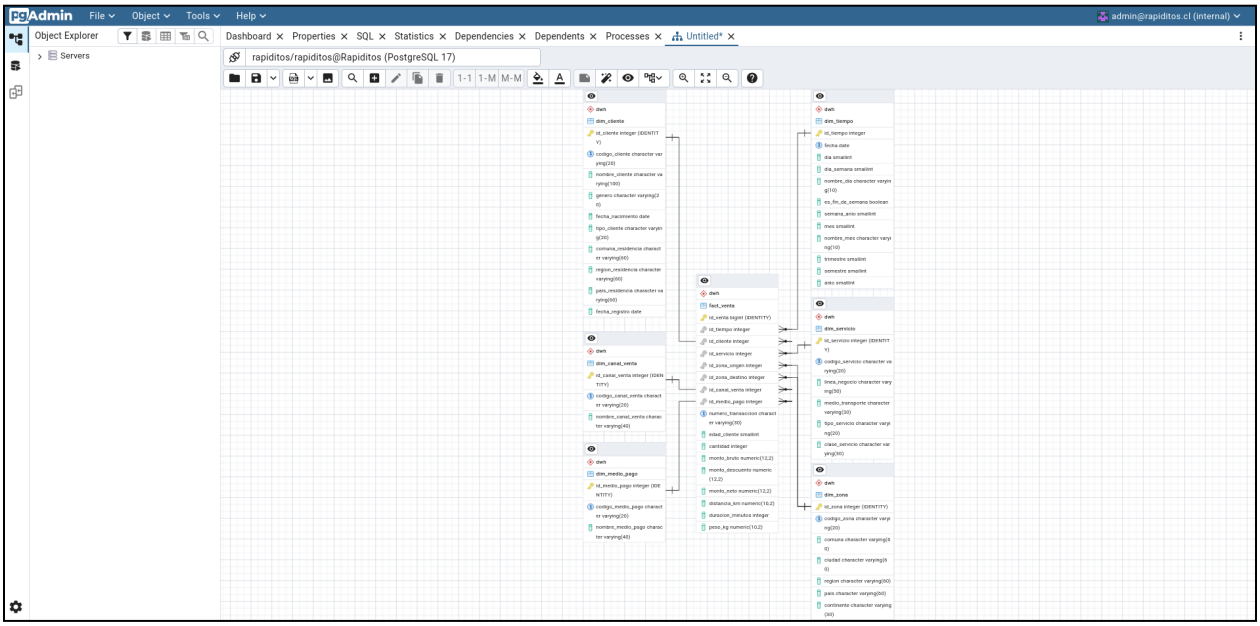
5.4. Entorno de Ejecución

El Datawarehouse corre en PostgreSQL 17.11 dentro de un contenedor Docker, junto con pgAdmin 4 (Figura 5.4). El esquema estrella creado por el script se visualiza en el ERD Tool de pgAdmin (Figura 5.5).

Figura 5.4: Salida de `docker compose ps`: PostgreSQL 17 y pgAdmin activos

```
hiro@hiro:~/Documentos/BI/proyecto-bi$ docker compose ps
NAME                   IMAGE                  COMMAND                  SERVICE    CREATED        STATUS                  PORTS
rapiditos-pgadmin-1    dpage/pgadmin4:9.18.0  "/entrypoint.sh"         pgadmin    29 hours ago   Up 4 minutes            443/tcp, 127.0.0.1:5050->80/tcp
rapiditos-postgres-1   postgres:17.11         "docker-entrypoint.s…"   postgres   29 hours ago   Up 4 minutes (healthy)  127.0.0.1:5432->5432/tcp
```

Figura 5.5: Esquema estrella en el ERD Tool de pgAdmin



5.5. Verificación del Datawarehouse

El resultado del script se verificó de tres formas: con las consultas de `sql/99_validacion.sql`, con las 29 pruebas automáticas de `tests/test_ddl.py` y ejecutando el script dos veces seguidas, para comprobar que es idempotente. La Tabla 5.2 resume los elementos del esquema `dwh` consultados en el catálogo de PostgreSQL.

Tabla 5.2: Elementos del esquema `dwh` verificados en el catálogo

Elemento	**Esperado según el diseño**	**Obtenido**
Tablas (1 de hechos y 6 dimensiones)	7	7
Columnas	58	58
Llaves primarias	7	7
Restricciones `UNIQUE`	7	7
Llaves foráneas (todas en `fact_venta`)	7	7
Restricciones `CHECK`	20	20
Índices sobre llaves foráneas	7	7
Comentarios (esquema, tablas y columnas)	66	66
Restricciones o índices fuera de la norma de nomenclatura	0	0
Filas cargadas (Avance N°1, sin datos)	0	0

Nota. Consultas ejecutadas en PostgreSQL 17.11 el 05-11-2026.

Las pruebas automáticas crean una base de prueba aparte, ejecutan el script dos veces y comparan la base resultante con el diseño de `docs/modelo_dimensional.md`: tablas, columnas, tipos, llaves, reglas e índices. También intentan insertar datos inválidos, como un monto neto que no cuadra con el bruto o una venta con un cliente inexistente, y comprueban que la base los rechaza. Las 29 pruebas se aprobaron.

Las figuras siguientes son resultados de `sql/99_validacion.sql`. La Figura 5.6 muestra las siete tablas del esquema con su comentario. La Figura 5.7 muestra las siete llaves foráneas de `fact_venta`, todas con nombre normado y regla `RESTRICT`. La Figura 5.8 confirma la estrella: solo la tabla de hechos tiene llaves foráneas salientes, y `dim_zona` es referenciada dos veces (origen y destino). La Figura 5.9 lista las reglas de integridad, y la Figura 5.10 muestra que ninguna restricción ni índice queda fuera de la norma de nomenclatura.

Figura 5.6: `sql/99_validacion.sql`: tablas del esquema **`dwh`**

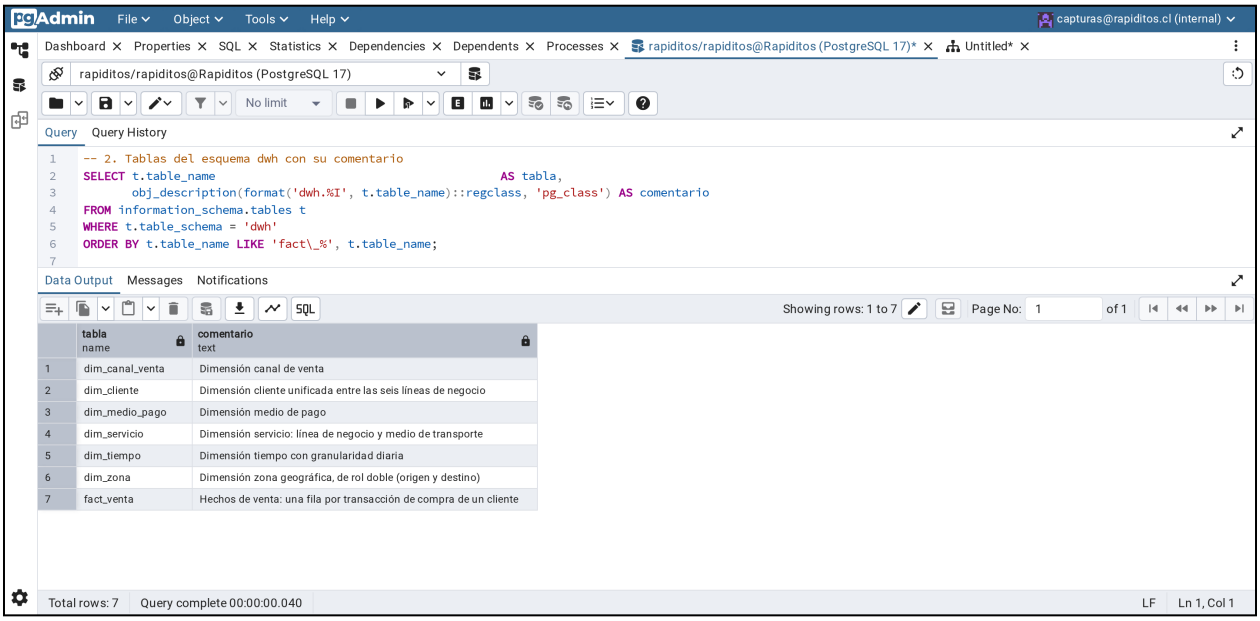


Figura 5.7: `sql/99_validacion.sql`: llaves foráneas de la tabla de hechos

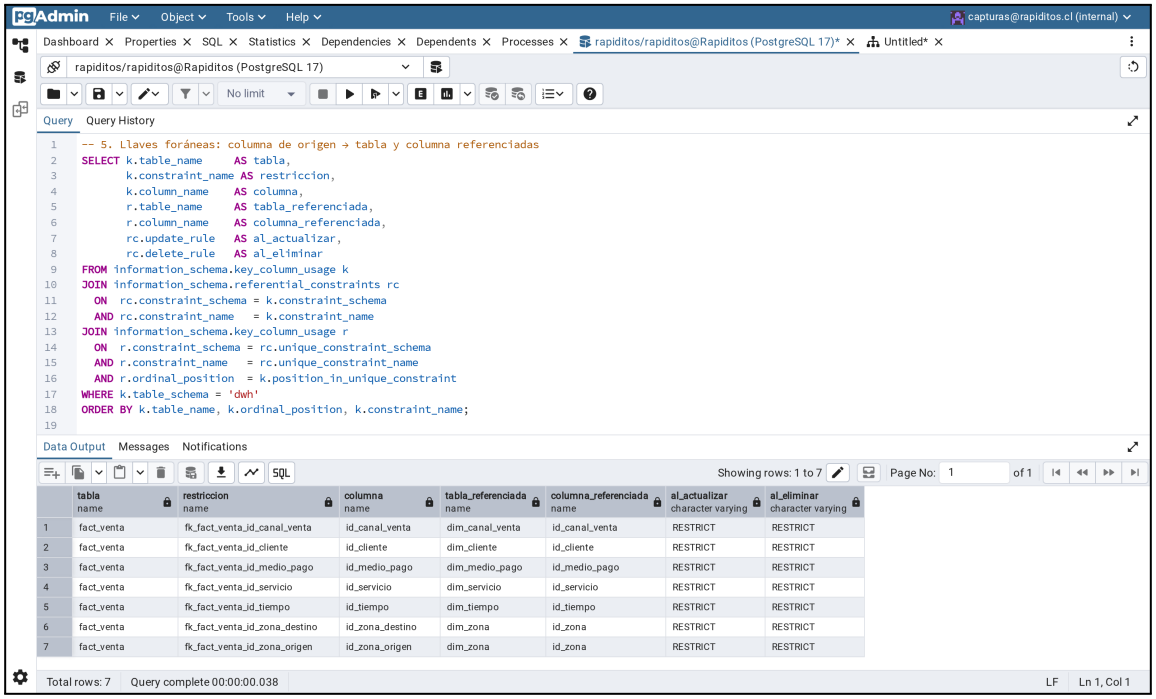


Figura 5.8: `sql/99_validacion.sql`: resumen de columnas y llaves por tabla

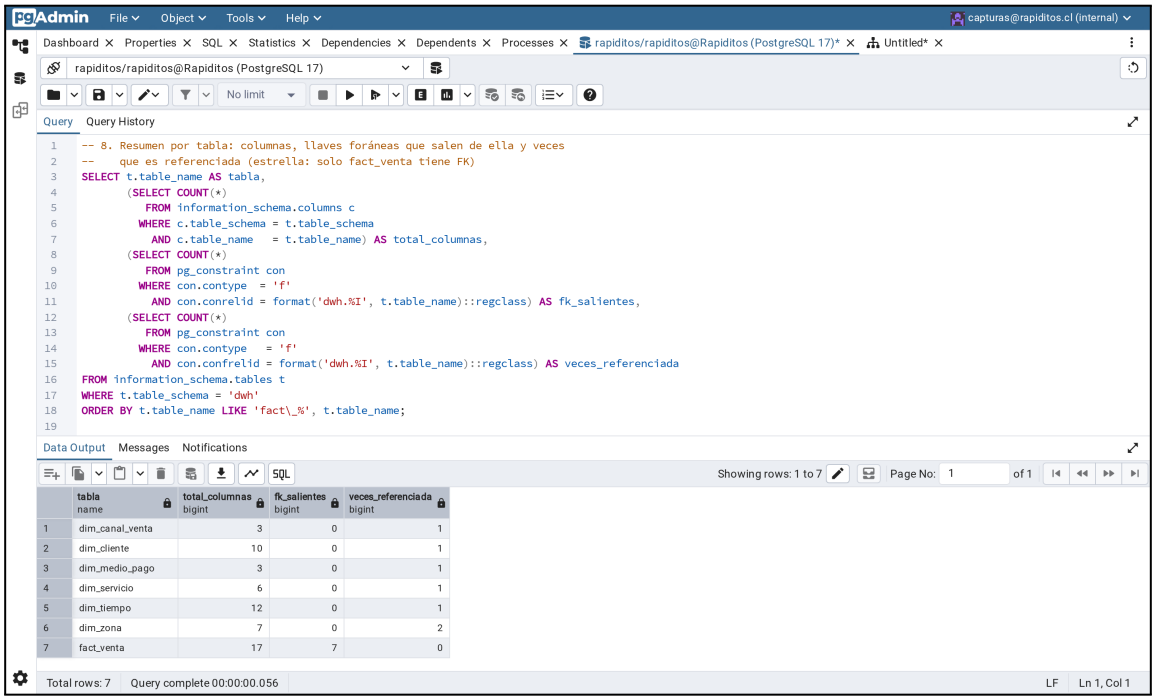


Figura 5.9: `sql/99_validacion.sql`: restricciones `CHECK`

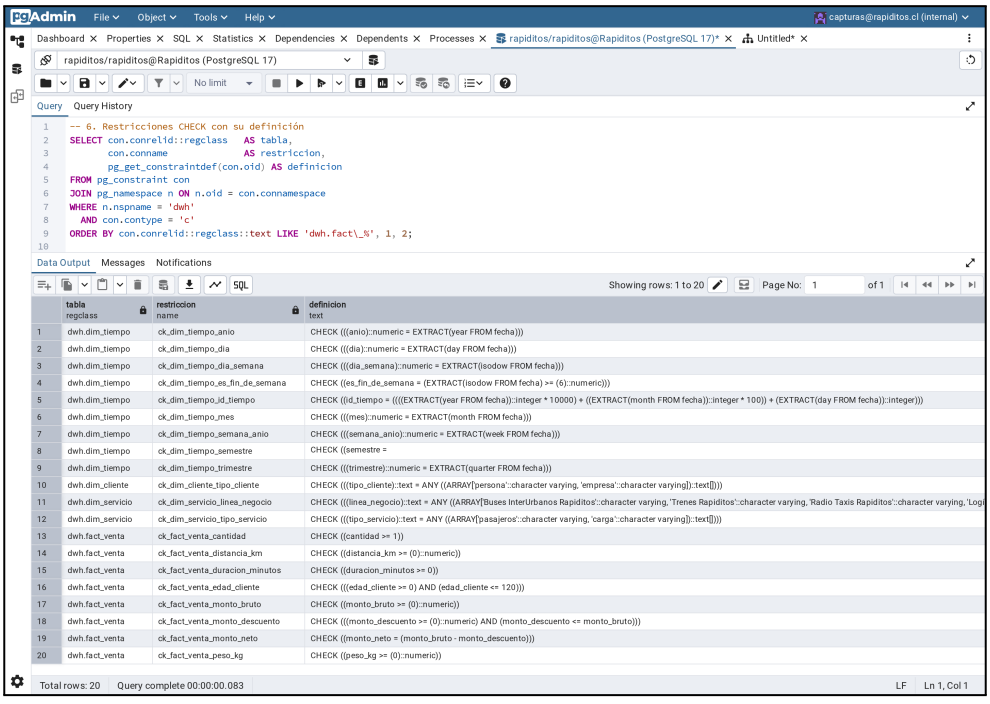
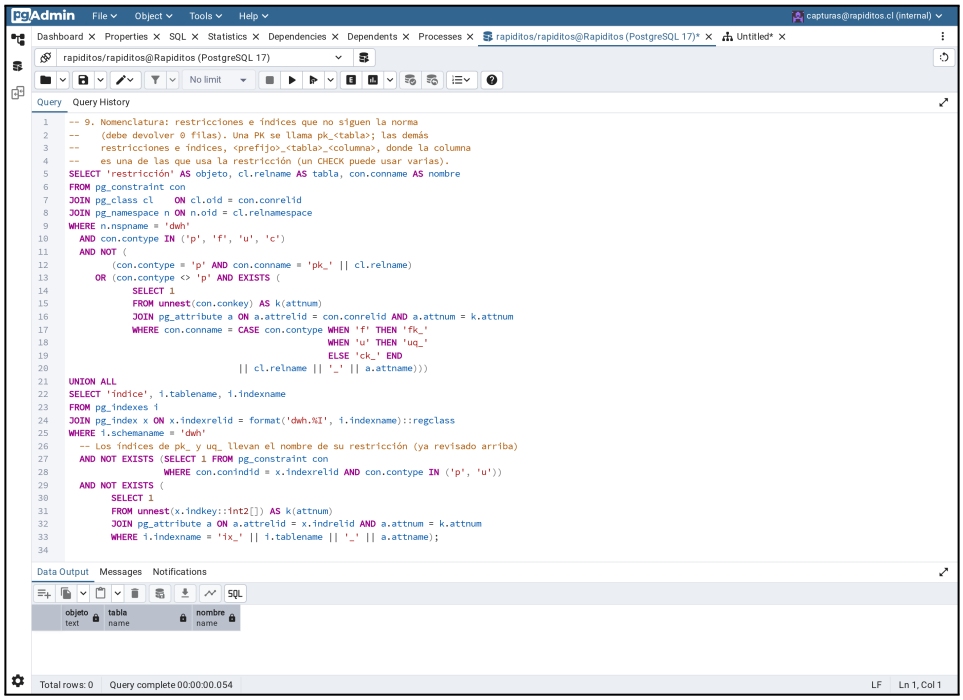


Figura 5.10: `sql/99_validacion.sql`: control de nomenclatura (0 filas fuera de norma)



6. Bibliografía

[1] Kimball, Ralph y Margy Ross: *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley, 3ª edición, 2013.

[2] Instituto Nacional de Estadísticas: *Estructura del transporte interurbano de pasajeros*. Estadísticas por tema, comercio y servicios. https://www.ine.gob.cl/estadisticas-por-tema/comercio-y-servicios/estructura-del-transporte-interurbano-de-pasajeros, visitado el 5 de noviembre de 2026.

[3] Empresa de los Ferrocarriles del Estado: *Memoria anual 2023*. Documento PDF, 2024. https://www.efe.cl/wp-content/uploads/2024/08/Memoria-EFE-Central-2023-v.pdf, visitado el 5 de noviembre de 2026.

[4] Junta de Aeronáutica Civil: *Sitio web institucional*. https://www.jac.gob.cl, visitado el 5 de noviembre de 2026.

[5] Instituto Nacional de Estadísticas: *Primeros resultados del Censo 2024: 18.480.432 personas fueron censadas en Chile, manteniéndose la tendencia de envejecimiento de la población*. Nota de prensa del 27 de marzo, 2025. https://www.ine.gob.cl/sala-de-prensa/prensa/general/noticia/2025/03/27/primeros-resultados-del-censo-2024-18.480.432-personas-fueron-censadas-en-chile-manteni%C3%A9ndose-la-tendencia-de-envejecimiento-de-la-poblaci%C3%B3n, visitado el 5 de noviembre de 2026.

[6] Instituto Nacional de Estadísticas: *INE publica base de datos del Censo de Población y Vivienda 2024*. Nota de prensa del 4 de diciembre, 2025. https://www.ine.gob.cl/sala-de-prensa/prensa/general/noticia/2025/12/04/ine-publica-base-de-datos-del-censo-de-poblaci%C3%B3n-y-vivienda-2024, visitado el 5 de noviembre de 2026.

[7] Agência Nacional de Transportes Terrestres: *Monitriip: Bilhetes de Passagem*. Conjunto de datos, Portal de Dados Abertos da ANTT. https://dados.antt.gov.br/dataset/monitriip-bilhetes-de-passagem, visitado el 5 de noviembre de 2026.

[8] Maven Analytics: *UK Train Rides*. Conjunto de datos, Maven Analytics Data Playground. `https://mavenanalytics.io/data-playground/uk-train-rides`, visitado el 5 de noviembre de 2026.

[9] NYC Taxi & Limousine Commission: *TLC Trip Record Data*. Conjunto de datos. `https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page`, visitado el 5 de noviembre de 2026.

[10] Olist: *Brazilian E-Commerce Public Dataset by Olist*. Conjunto de datos, Kaggle. `https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce`, visitado el 5 de noviembre de 2026.

[11] Bureau of Transportation Statistics: *Airline Origin and Destination Survey (DB1B)*. Conjunto de datos, U.S. Department of Transportation. `https://www.bts.gov/topics/airlines-and-airports/origin-and-destination-survey-data`, visitado el 5 de noviembre de 2026.

[12] Gironews: *[Nota sobre Delivery Center y la publicación de su base de pedidos]*. Varejo digital, 2021. `https://gironews.com/varejo-digital/marketplaces-no-delivery-65728/`, visitado el 5 de noviembre de 2026, Título descriptivo.

[13] lore5g: *Delivery Center Dataset Brazil*. Repositorio de GitHub con una copia del dataset «Delivery Center: Food & Goods orders». `https://github.com/lore5g/Delivery-Center-Dataset-Brazil`, visitado el 5 de noviembre de 2026.

[14] Constante, F., F. Silva y A. Pereira: *DataCo Smart Supply Chain for Big Data Analysis*. Conjunto de datos, versión 5, Mendeley Data, 2019. `https://doi.org/10.17632/8gx2fvg2k6.5`, visitado el 5 de noviembre de 2026.

[15] Banco Central de Chile: *Sitio web institucional*. `https://www.bcentral.cl`, visitado el 5 de noviembre de 2026.

[16] The Apache Software Foundation: *Apache Hop User Manual*. `https://hop.apache.org/manual/latest/`, visitado el 5 de noviembre de 2026.

[17] PostgreSQL Global Development Group: *PostgreSQL 17 Documentation*. `https://www.postgresql.org/docs/17/`, visitado el 5 de noviembre de 2026.

[18] pgAdmin Development Team: *pgAdmin 4 Documentation*. `https://www.pgadmin.org/docs/pgadmin4/latest/`, visitado el 5 de noviembre de 2026.

[19] Docker: *Docker Compose Documentation*. `https://docs.docker.com/compose/`, visitado el 5 de noviembre de 2026.

[20] Cube Dev: *Cube Documentation*. `https://cube.dev/docs`, visitado el 5 de noviembre de 2026.

[21] Streamlit: *Streamlit Documentation*. `https://docs.streamlit.io/`, visitado el 5 de noviembre de 2026.

7. Anexos

7.1. Anexo A: Diccionario de Datos

Este anexo describe cada columna del esquema `dwh`: tipo de dato, si admite nulos, las llaves y reglas en que participa (PK: llave primaria; FK: llave foránea; UQ: valor único; CK: regla `CHECK`) y su descripción. Las descripciones son los comentarios `COMMENT ON` del propio script, y el anexo se genera desde el catálogo de PostgreSQL con `herramientas/generar_diccionario.py`, por lo que coincide con la base creada.

7.1.1. Tabla `fact_venta`

Hechos de venta: una fila por transacción de compra de un cliente. **Origen:** Transacciones de las seis fuentes públicas de la Sección 2 (una fila por pasaje, viaje, envío o pedido), asignadas a clientes sintéticos.

Tabla 7.1: Diccionario de datos de `fact_venta`

Columna	Tipo	Nulo	Clave	Descripción
`id_venta`	BIGINT	No	PK	PK. Llave sustituta
`id_tiempo`	INTEGER	No	FK	FK a dim_tiempo (fecha de la compra)
`id_cliente`	INTEGER	No	FK	FK a dim_cliente
`id_servicio`	INTEGER	No	FK	FK a dim_servicio
`id_zona_origen`	INTEGER	No	FK	FK a dim_zona, rol origen
`id_zona_destino`	INTEGER	No	FK	FK a dim_zona, rol destino
`id_canal_venta`	INTEGER	No	FK	FK a dim_canal_venta
`id_medio_pago`	INTEGER	No	FK	FK a dim_medio_pago
`numero_transaccion`	VARCHAR(30)	No	UQ	Dimensión degenerada: id del sistema de origen, prefijado por línea
`edad_cliente`	SMALLINT	Sí	CK	Edad al momento de la compra (no aditiva; NULL para empresas)
`cantidad`	INTEGER	No	CK	Medida aditiva: pasajes, bultos o unidades
`monto_bruto`	NUMERIC(12,2)	No	CK	Medida aditiva: monto antes de descuento (CLP)
`monto_descuento`	NUMERIC(12,2)	No	CK	Medida aditiva: descuento aplicado (CLP)
`monto_neto`	NUMERIC(12,2)	No	CK	Medida aditiva: monto pagado (CLP) = bruto - descuento
`distancia_km`	NUMERIC(10,2)	Sí	CK	Medida aditiva: distancia recorrida en km
`duracion_minutos`	INTEGER	Sí	CK	Medida aditiva: duración del servicio en minutos
`peso_kg`	NUMERIC(10,2)	Sí	CK	Medida aditiva: peso en kg (solo logística y delivery)

7.1.2. Tabla `dim_tiempo`

Dimensión tiempo con granularidad diaria. **Origen:** Se genera con un script SQL (`generate_series`) para 2023 a 2025; no requiere fuente externa.

Tabla 7.2: Diccionario de datos de `dim_tiempo`

Columna	Tipo	Nulo	Clave	Descripción
`id_tiempo`	INTEGER	No	PK, CK	PK. Fecha en formato AAAAMMDD (ej. 20250314)
`fecha`	DATE	No	UQ	Fecha calendario
`dia`	SMALLINT	No	CK	Día del mes (1-31)
`dia_semana`	SMALLINT	No	CK	1 = lunes … 7 = domingo (ISO)
`nombre_dia`	VARCHAR(10)	No		Lunes … Domingo
`es_fin_de_semana`	BOOLEAN	No	CK	TRUE si es sábado o domingo
`semana_anio`	SMALLINT	No	CK	Semana ISO del año (1-53)
`mes`	SMALLINT	No	CK	Mes (1-12)
`nombre_mes`	VARCHAR(10)	No		Enero … Diciembre
`trimestre`	SMALLINT	No	CK	Trimestre (1-4)
`semestre`	SMALLINT	No	CK	Semestre (1-2)
`anio`	SMALLINT	No	CK	Año

7.1.3. Tabla `dim_cliente`

Dimensión cliente unificada entre las seis líneas de negocio. **Origen:** Sintética: clientes residentes en Chile, con la distribución por comuna, sexo y edad del Censo de Población y Vivienda 2024.

Tabla 7.3: Diccionario de datos de `dim_cliente`

Columna	Tipo	Nulo	Clave	Descripción
`id_cliente`	INTEGER	No	PK	PK. Llave sustituta
`codigo_cliente`	VARCHAR(20)	No	UQ	Llave natural (RUT o documento, unificado entre líneas)
`nombre_cliente`	VARCHAR(100)	No		Nombre o razón social
`genero`	VARCHAR(20)	Sí		Género declarado (NULL para empresas)
`fecha_nacimiento`	DATE	Sí		Fecha de nacimiento (NULL para empresas)
`tipo_cliente`	VARCHAR(20)	No	CK	persona o empresa
`comuna_residencia`	VARCHAR(60)	Sí		Comuna de residencia
`region_residencia`	VARCHAR(60)	Sí		Región de residencia
`pais_residencia`	VARCHAR(60)	No		País de residencia
`fecha_registro`	DATE	Sí		Fecha de alta como cliente

7.1.4. Tabla `dim_servicio`

Dimensión servicio: línea de negocio y medio de transporte. **Origen:** Catálogo de los servicios de cada línea de negocio, con los atributos que informa cada fuente (tipo y clase de servicio, medio de transporte).

Tabla 7.4: Diccionario de datos de `dim_servicio`

Columna	Tipo	Nulo	Clave	Descripción
`id_servicio`	INTEGER	No	PK	PK. Llave sustituta
`codigo_servicio`	VARCHAR(20)	No	UQ	Llave natural
`linea_negocio`	VARCHAR(50)	No	CK	Una de las 6 líneas de negocio del enunciado (nombre literal)
`medio_transporte`	VARCHAR(30)	No		bus, tren, taxi, camión, avión, moto/bicicleta
`tipo_servicio`	VARCHAR(20)	No	CK	pasajeros o carga
`clase_servicio`	VARCHAR(30)	No		Ej. económica, ejecutiva, express, estándar

7.1.5. Tabla `dim_zona`

Dimensión zona geográfica, de rol doble (origen y destino). **Origen:** Tablas de zonas de cada fuente (zonas de taxi de Nueva York, puntos de ANTT, estaciones del Reino Unido, aeropuertos de EE.UU.) y comunas de Chile del Censo 2024.

Tabla 7.5: Diccionario de datos de `dim_zona`

Columna	Tipo	Nulo	Clave	Descripción
`id_zona`	INTEGER	No	PK	PK. Llave sustituta
`codigo_zona`	VARCHAR(20)	No	UQ	Llave natural
`comuna`	VARCHAR(60)	No		Comuna (No aplica en destinos internacionales)
`ciudad`	VARCHAR(60)	No		Ciudad
`region`	VARCHAR(60)	No		Región
`pais`	VARCHAR(60)	No		País
`continente`	VARCHAR(30)	No		Continente

7.1.6. Tabla `dim_canal_venta`

Dimensión canal de venta. **Origen:** Canales de compra informados por las fuentes, normalizados a un catálogo común.

Tabla 7.6: Diccionario de datos de `dim_canal_venta`

Columna	Tipo	Nulo	Clave	Descripción
`id_canal_venta`	INTEGER	No	PK	PK. Llave sustituta
`codigo_canal_venta`	VARCHAR(20)	No	UQ	Llave natural
`nombre_canal_venta`	VARCHAR(40)	No		app, web, boletería, call center, convenio empresa

7.1.7. Tabla `dim_medio_pago`

Dimensión medio de pago. **Origen:** Tipos de pago informados por las fuentes, normalizados a un catálogo común.

Tabla 7.7: Diccionario de datos de `dim_medio_pago`

Columna	Tipo	Nulo	Clave	Descripción
`id_medio_pago`	INTEGER	No	PK	PK. Llave sustituta
`codigo_medio_pago`	VARCHAR(20)	No	UQ	Llave natural
`nombre_medio_pago`	VARCHAR(40)	No		efectivo, débito, crédito, transferencia, billetera digital

7.2. Anexo B: Reproducción del Entorno

7.2.1. Ejecución de los Scripts en Cualquier Equipo

Los scripts `01_ddl.sql` y `99_validacion.sql` son autónomos: no leen ni escriben archivos, no dependen de otros archivos y se ejecutan desde cualquier carpeta, en Windows, Linux o macOS. Solo requieren PostgreSQL 17 y una base de datos existente; se recomienda una base nueva, porque `01_ddl.sql` borra y vuelve a crear el esquema `dwh`.

1. Crear una base de datos UTF-8, por ejemplo `rapiditos`.
2. En pgAdmin, abrir `01_ddl.sql` en el Query Tool de esa base y ejecutarlo completo con F5. Con `psql`, desde la carpeta en que estén los scripts:

```
psql -U <usuario> -d rapiditos -v ON_ERROR_STOP=1 -f 01_ddl.sql
psql -U <usuario> -d rapiditos -P pager=off -f 99_validacion.sql
```

3. Comparar el resultado de `99_validacion.sql` con la Tabla 5.2: la última consulta debe devolver cero filas.

Para Windows se entrega además la guía `guia_ejecucion_windows.pdf`, que detalla paso a paso la instalación de PostgreSQL y pgAdmin, la ejecución con `psql` y una alternativa con Docker Desktop que no requiere instalar PostgreSQL ni ningún otro archivo del proyecto.

7.2.2. Entorno de Desarrollo del Grupo

El grupo trabaja con el repositorio completo del proyecto, que define PostgreSQL y pgAdmin en `docker-compose.yml` e incluye las pruebas automáticas. Estos pasos requieren ese repositorio y se ejecutan desde su raíz, igual en Linux y en Windows; solo cambian la copia del archivo `.env` y la activación del entorno virtual de Python, indicadas en los comentarios.

1. Preparar las credenciales y levantar PostgreSQL y pgAdmin. En `.env` se definen las contraseñas de ambos servicios.

```
cp .env.example .env      # Windows: copy .env.example .env
docker compose up -d
docker compose ps         # ambos servicios deben aparecer activos
```

2. Crear el modelo multidimensional; pgAdmin queda en `http://localhost:5050`.

```
docker compose exec postgres psql -U rapiditos -d rapiditos \
    -v ON_ERROR_STOP=1 -f /sql/01_ddl.sql
```

3. Crear el entorno de Python y ejecutar las pruebas automáticas, que usan una base de prueba aparte y nunca modifican `rapiditos`.

```
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
pytest -v
```

4. Opcionalmente, regenerar el diccionario de datos del Anexo A y las capturas del informe.

```
python herramientas/generar_diccionario.py
python -m playwright install chromium   # una sola vez
python herramientas/capturar_pgadmin.py
```

7.3. Anexo C: Repositorio del Proyecto

El código fuente completo del proyecto está publicado en un repositorio público de GitHub:

`https://github.com/hirojika/datawarehouse-transportes-rapiditos`

El repositorio reúne los scripts del Datawarehouse, las pruebas automáticas, las herramientas de apoyo, la documentación del diseño y las fuentes de este informe, de modo que cualquier persona puede revisar o reproducir el trabajo. Cada entrega se registra en Git con un commit descriptivo, y el archivo `docs/bitacora.md` documenta, con fecha, cada decisión, cambio y verificación realizada durante el proyecto. La Tabla 7.8 describe su contenido.

Tabla 7.8: Contenido del repositorio del proyecto

Carpeta o archivo	**Contenido**
`sql/`	Scripts del Datawarehouse: `01_ddl.sql` (creación del modelo estrella) y `99_validacion.sql` (verificación)
`docs/`	Diseño del modelo dimensional (`modelo_dimensional.md`, fuente de verdad del diseño), fuentes de información, bitácora, diagrama del esquema estrella y capturas del proceso
`informe/`	Fuentes LaTeX de este informe y de la guía de ejecución en Windows, con las referencias en `referencias.bib`
`tests/`	Pruebas automáticas del script del Datawarehouse (pytest)
`herramientas/`	Captura automatizada de pgAdmin, generación del diccionario de datos (Anexo A) y armado del paquete de entrega
`docker-compose.yml`, `docker/`	Definición de los servicios PostgreSQL 17 y pgAdmin 4, con versiones fijas
`requirements.txt`, `.env.example`	Dependencias de Python con versiones fijas y plantilla de configuración de la conexión
`etl/`, `dashboard/`, `data/`	Carpetas reservadas para el proyecto de Apache Hop, el dashboard en Streamlit y los datos de origen, que se desarrollan en los Avances N°2 y N°3

Por seguridad y derechos de autor, el repositorio no incluye las credenciales de conexión (archivo `.env`, que cada integrante crea a partir de `.env.example`) ni el material del curso entregado por el profesor. Para obtener una copia local se usa:

```
git clone https://github.com/hirojika/datawarehouse-transportes-rapiditos.git
```

Los pasos para levantar el entorno a partir de esa copia están en el Anexo B (Sección 7.2).